



Busca de palavras em textos

1 Contexto

O Dr. Alcebíades é um advogado das antigas. Ao longo de sua carreira, ele já atuou em diversos processos nas áreas do Direito Civil, Penal, Trabalhista e de Família, acumulando muitos casos de sucesso e uma grande coleção de documentos jurídicos.

No entanto, Alcebíades é muito organizado com seus documentos, mas pouco habilidoso com tecnologia. Ele utiliza o computador praticamente apenas para escrever suas petições, pareceres e demais peças processuais em um editor de texto. Seus documentos estão espalhados entre o computador, diversos pendrives e discos externos.

Quando começa um novo processo, frequentemente se lembra de um argumento ou de uma fundamentação jurídica utilizada em um caso anterior, mas não consegue se lembrar em qual documento ela está. Ele gostaria que seu computador permitisse realizar buscas por palavras e retornasse os documentos nos quais essas palavras aparecem.

Como Alcebíades não entende muito de programação, seu sobrinho sugeriu que ele desenvolvesse um pequeno sistema de busca baseado em um índice invertido. Dessa forma, seria possível associar cada palavra aos documentos em que ela aparece e localizar rapidamente os arquivos relacionados a uma determinada consulta.

Alcebíades não entendeu muito bem como um índice invertido poderia ajudá-lo, mas decidiu seguir a sugestão e elaborar um esquema para construir seu índice invertido.

2 Objetivo

Desenvolver uma aplicação capaz de indexar palavras presentes em diversos arquivos de texto utilizando uma Árvore Digital Ternária. A partir do índice construído, o sistema deverá permitir realizar pesquisas por palavras e consultas utilizando operadores lógicos: E, OU e NAO.

O objetivo principal é aplicar os conceitos de árvores, busca, indexação, processamento de strings, conjuntos e análise de complexidade na construção de um mecanismo de busca simplificado.

3 O problema

O Dr. Alcebíades precisa de um sistema de busca de documentos que permita localizar rapidamente arquivos a partir de palavras-chave. O sistema deverá receber uma coleção de documentos, representados, por simplicidade, como arquivos de texto, e construir uma estrutura de dados que permita realizar consultas de forma eficiente.



Dada uma palavra, o sistema deverá ser capaz de informar quais documentos contêm essa palavra.

Por exemplo, suponha que existam os seguintes documentos:

```
processo_001.txt  
processo_002.txt  
processo_003.txt  
processo_004.txt
```

E que a palavra contrato apareça nos documentos:

```
processo_001.txt  
processo_003.txt  
processo_004.txt
```

Ao realizar a consulta:

contrato

o sistema deverá retornar os documentos correspondentes.

O sistema também deverá permitir consultas contendo mais de uma palavra, possibilitando buscas como:

contrato E responsabilidade NAO multa

Nesse caso, deverão ser considerados os documentos que atendam aos critérios definidos pelo sistema de busca. Neste caso, o sistema deve mostrar os documentos em que apareçam as palavras contrato e responsabilidade e não contenha multa.

4 Índice invertido

Um índice invertido é uma estrutura de dados que associa cada palavra aos documentos em que ela aparece.

Por exemplo, considerando os documentos:

```
arquivo1.txt:  
  contrato de compra
```

```
arquivo2.txt:  
  contrato de aluguel
```

```
arquivo3.txt:  
  compra e venda
```



O índice invertido poderia ser:

compra: [arquivo1.txt, arquivo3.txt]

contrato: [arquivo1.txt, arquivo2.txt]

aluguel: [arquivo2.txt]

venda: [arquivo3.txt]

Assim, para uma consulta como "contrato", o sistema não precisa percorrer todos os documentos procurando pela palavra. Basta consultar o índice e obter diretamente:

contrato: [arquivo1.txt, arquivo2.txt]

Em outras palavras:

Um índice invertido é uma estrutura de dados que, para cada palavra, mantém a lista de documentos nos quais essa palavra ocorre.

O termo "invertido" vem justamente da forma como a informação é organizada: em vez de partir de documento → palavras, o índice organiza as informações como palavra → documentos.

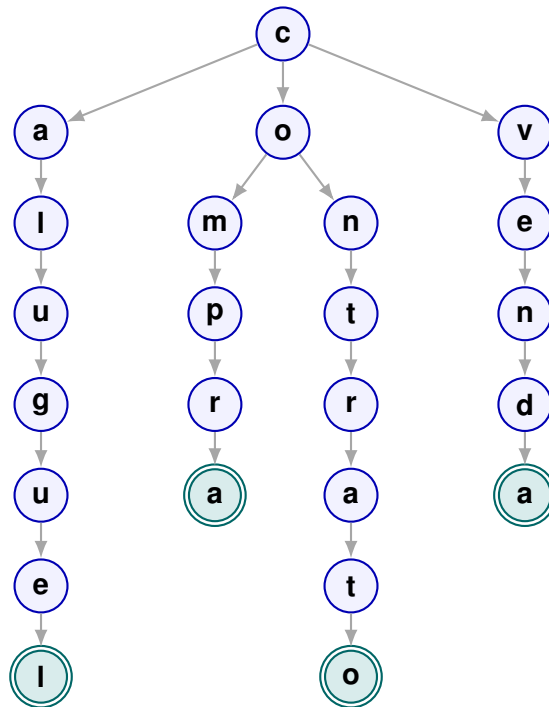
5 Árvore Ternária Digital

Uma Árvore Digital Ternária (*trie*) é uma estrutura de dados na qual cada nó armazena um caractere e possui três referências: esquerda, meio e direita. O ponteiro da esquerda aponta para caracteres menores (na ordem lexicográfica) que o caractere armazenado no nó atual. O ponteiro do meio aponta para o próximo caractere da palavra sendo buscada ou inserida. O ponteiro para a direita aponta para nós com caracteres maiores que o caractere do nó atual. Além disso, adicione uma flag que indica o fim da palavra, que é um valor booleano para indicar que aquele conclui uma palavra válida.

Para as palavras:

- compra
- contrato
- aluguel
- venda

Temos a seguinte árvore:



6 Estrutura mínima do nó

O nó da árvore deverá possuir, no mínimo, informações equivalentes a:

```
classe No
```

```
caractere: char
```

```
esquerda: No
```

```
meio: No
```

```
direita: No
```

```
fimDaPalavra: Boolean
```

```
arquivos associados: Lista<String>
```



6.1 Como funciona a busca e inserção?

Para buscar ou inserir uma palavra (por exemplo, "compra"):

Compara-se a primeira letra ('c') com o caractere do nó atual.

Se for menor: Segue pelo ponteiro da esquerda (sem avançar na letra da palavra).

Se for maior: Segue pelo ponteiro da direita (sem avançar na letra da palavra).

Se for igual: Avança para a próxima letra da palavra ('o') e desce pelo ponteiro do meio.

Repete-se o processo até encontrar o fim da palavra ou atingir um ponteiro nulo.

6.2 Construção do índice

O programa deverá permitir a indexação de vários arquivos.

Para cada arquivo, o programa deverá:

1. abrir o arquivo;
2. ler seu conteúdo;
3. identificar as palavras;
4. normalizar as palavras;
5. inserir cada palavra na Árvore Digital Ternária;
6. associar o arquivo à palavra.

A árvore deverá ser construída uma única vez, antes da realização das consultas.

Após o índice ser construído, o programa deverá permitir realizar várias consultas.

7 Normalização das palavras

O programa deverá adotar uma política consistente de normalização.

No mínimo:

7.1 Maiúsculas e minúsculas

As palavras:



Venda
VENDA
venda

deverão ser consideradas iguais.

7.2 Pontuação

As palavras:

venda,
venda.
venda;

deverão ser consideradas iguais a:

venda

A normalização deverá ser aplicada tanto durante a indexação quanto durante a pesquisa.

8 Precedência de operadores e uso de parênteses

Os operadores deverão obedecer à seguinte precedência:

1. NAO;
2. E;
3. OU.

Por exemplo a consulta: compra OU venda E casa

Deverá ser interpretado como: compra OU (venda E casa)

A implementação de parênteses será considerada uma funcionalidade adicional.

Por exemplo: (casa OU apartamento) E NAO terreno



9 Interface do programa

Uma possível execução:

```
=====
                INDICE INVERTIDO
=====

Arquivos encontrados: 5

Construindo indice...
Indice construido com sucesso.

Digite uma consulta:
> compra

Arquivos encontrados:
entrada_01.txt
entrada_03.txt

Digite uma consulta:
> compra E casa

Arquivos encontrados:
entrada_01.txt
entrada_03.txt

Digite uma consulta:
> casa E apartamento E NAO terreno

Arquivos encontrados:
entrada_01.txt
entrada_03.txt

Digite uma consulta:
> sair

Programa encerrado.
```

A interface poderá ser diferente, desde que todas as funcionalidades sejam disponibilizadas.



10 Requisitos obrigatórios

O programa deverá obrigatoriamente:

- utilizar uma Árvore Digital Ternária;
- implementar os nós da árvore;
- possuir os ponteiros esquerda, meio e direita;
- armazenar caracteres nos nós;
- indicar o fim das palavras;
- implementar inserção;
- implementar busca;
- construir o índice a partir de arquivos;
- associar palavras aos arquivos;
- evitar arquivos duplicados na associação de uma palavra;
- realizar pesquisa simples;
- implementar E;
- implementar OU;
- implementar NAO;
- aceitar consultas com várias palavras;
- tratar palavras inexistentes;
- normalizar palavras;
- apresentar os resultados de forma organizada.

11 Restrições

Não será permitido utilizar como estrutura principal do índice:

HashMap
Dictionary
Map
HashSet
Set
TreeMap



ou estruturas equivalentes.

Também não será permitido substituir a **Árvore Digital Ternária** por:

Trie
Árvore Binária de Busca tradicional
AVL
Red-Black Tree

O objetivo específico deste trabalho é implementar e utilizar uma **Árvore Digital Ternária**.

Estruturas auxiliares poderão ser utilizadas quando justificadas, por exemplo, para armazenar a lista de arquivos associados a uma palavra.

12 Critérios de avaliação

O trabalho valerá **10,0 pontos**.

Critério	Pontos
Implementação correta da Árvore Digital Ternária	2,5
Inserção e busca de palavras	1,5
Construção do índice invertido	1,5
Operador E	0,75
Operador OU	0,75
Operador NAO	0,75
Consultas combinadas	0,50
Tratamento de entradas e palavras inexistentes	0,25
Relatório e análise de complexidade	1,0
Organização e qualidade do código	0,50
Total	10,0

Desonestidade acadêmica

Este trabalho deve ser feito individualmente ou em dupla. Evidentemente você pode “discutir” o material dado em classe, inclusive sobre os problemas e “dicas” para chegar às soluções, mas você deve ser responsável pela solução final e pelo desenvolvimento de seus programas. Portanto, cópias totais/parciais dos programas terão nota zero. Além disso, não serão aceitos projetos atrasados. Os casos de desonestidade serão tratados com rigor.